Hackers Hideout — Splunk Log Investigation Study Guide

Question by question derivation, straight from the exported event log CSV, with real executed queries and results

- This dataset is unusual: it is not one attack, it is two, chained together through a single pivot
- Phase 1 exploits a web LFI vulnerability to read the server's /etc/passwd file, Questions 1 through 5, and 9
- That file is exactly where Phase 2's SSH brute force gets the username it needs, confirmed directly in Question 9
- Without Phase 1's file exposure, Phase 2's attacker would have had no known account to target
- This guide follows all 13 questions in their original order, since that order already reflects the real investigative flow

Attacker	Target host	Data source
192.168.178.83	ironhack	Apache Access Log + Auth Logs, exported as logs_1.csv

---

UNDERSTANDING REX AND _RAW

- Several questions in this dataset need a technique called **rex**, because the fields a query would normally rely on (`clientip`, `status`, `user`, `pid`) were never automatically extracted for this custom sourcetype
- This section explains the tool once, so every rex query later in the guide makes sense on sight, even months from now

> **Key concept — _raw**
> - The complete, unprocessed text of a log line, exactly as Splunk received it, before any fields are pulled out of it
> - Every event has this field, always, no matter the sourcetype

> **Key concept — rex**
> - Short for "regular expression"
> - The command that reads through _raw text and pulls a piece of it out into a brand new, usable field
> - You are teaching Splunk a field it does not already know

The shape is always the same:

```
rex field=_raw "PATTERN(?<new_field_name>WHAT_TO_CAPTURE)MORE_PATTERN"
```

- Everything **outside** `(?<...>...)` is context, telling Splunk *where* in the line to look. It is not captured
- Everything **inside** `(?<...>...)` is what actually becomes the new field's value

> **Key concept — Memory tip — reading any rex pattern**
> - Almost every pattern in this guide reduces to one idea: a character class, followed by a quantifier, followed by a boundary
> - **Character class** — what kind of character are we grabbing? `\d` is a digit, `\S` is anything but a space, `[^\"]` is anything but a quote
> - **Quantifier** — how many of them? `+` always means one or more, keep going
> - **Boundary** — where do we stop? A literal character like `\]` or `\"`, or an anchor like `$` for end of line
> - Read any pattern left to right as: "grab this kind of character, keep grabbing, stop at this boundary"

Why this happened at all:

- This custom sourcetype (`Apache Access Log`) never had Splunk's standard web-log field extractions configured,

unlike a sourcetype literally named `access_combined`, which gets `clientip` / `status` automatically

- When a field silently does not exist, a filter like `status=200` matches zero events rather than raising an error, and `table pid` just renders an empty column
- Worth remembering as a general Splunk lesson, not just a fix for this one dataset

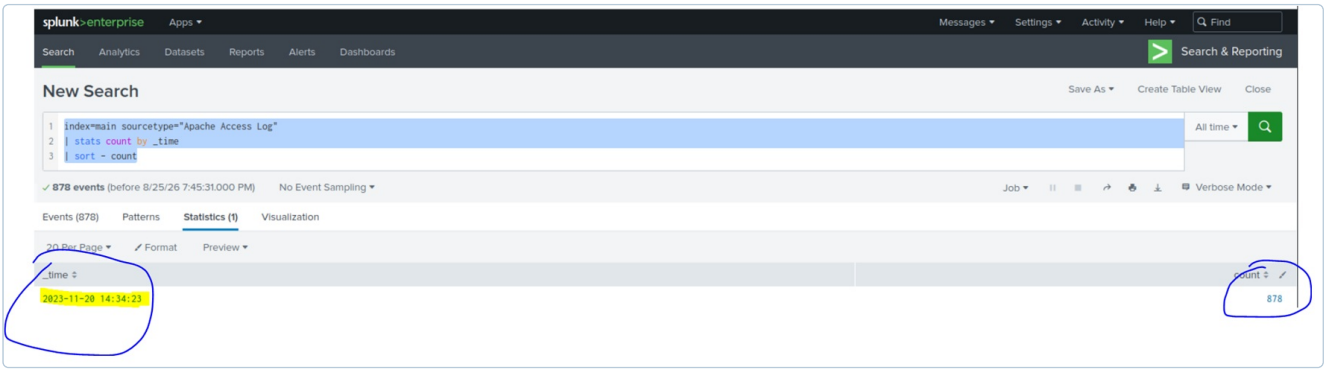
---

QUESTION 1

"*What is the technique used for the first attack?*"

> **Key concept — Brute force**
>
> - Repeatedly guessing values, directories, passwords, anything, at automated speed until one succeeds
> - The volume and speed is the signature, not what is being targeted

- **Decode:** this asks for a technique category, not a tool name (Q3) or a vulnerability name (Q2)
- **How this was actually found:** no SPL was run first. The event list was opened in Splunk's Search UI and scrolled through manually



stats count by _time, sorted descending. Executed query, real result: 878 events, timestamp 2023-11-20 14:34:23, count 878.

Three checks against the raw data, confirming the visual impression:

- **Volume and timing** — all 878 requests share the identical timestamp 14:34:23. No human triggers 878 requests in one second
- **Path diversity** — all 878 requested paths are distinct, zero repeats. This rules out a simple repeated request flood and points to systematic discovery instead
- **Source consistency** — one IP throughout, not a distributed flood

```
index=main sourcetype="Apache Access Log"
| stats count by _time
| sort - count
```

Field	Meaning
`sourcetype="Apache Access Log"`	Scopes to the web access log specifically
`stats count by _time`	Counts events per exact timestamp
`sort - count`	Surfaces the busiest timestamp first

> **Answer: brute force**
>
> - Confirmed against THM's official answer key
> - THM uses "brute force" as the umbrella term for automated enumeration generally, covering both directory fuzzing (here) and credential guessing (Q6), not credential guessing specifically

---

QUESTION 2

"*What is the name of the attack used by the technique intended above?*"

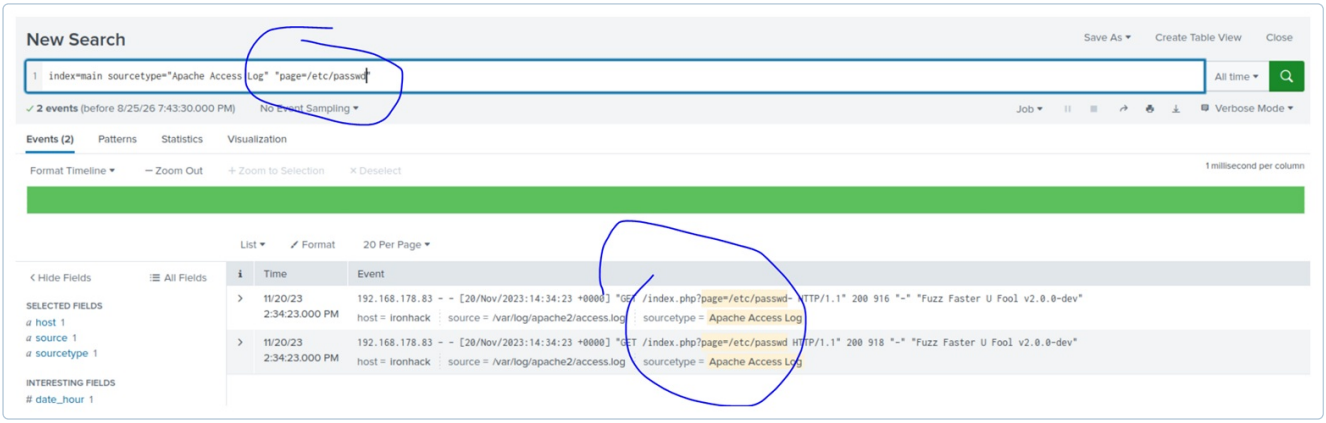
> **Key concept — LFI, Local File Inclusion**
>
> - A vulnerability where user-supplied input controls which file a server reads, letting an attacker pull arbitrary files off the filesystem instead of the page the app intended

How this actually works, step by step:

- Many web apps build pages from smaller reusable pieces: a header, a footer, and a content section that changes per page
- PHP has a built in function, `include()`, that means "open this file and paste its contents right here", like copy-paste run at request time
- The intended design: `include($_GET['page'])`, so a link like `index.php?page=about.php` loads that template and displays it
- The flaw: `include()` does not check what it is handed. It just takes whatever string sits in that parameter and treats it as a file path to open
- There is no built in restriction saying "only files ending in .php from this one folder", that check has to be written by the developer, and here it was not
- What the attacker actually did: sent `page=/etc/passwd` instead of `page=about.php`. The server's `include()` call opened `/etc/passwd` and dumped its contents into the response, exactly as it would for a legitimate template
- One line summary: LFI is a normal file loading feature with no guardrail on which files it is allowed to load

Not limited to PHP:

- Same pattern in Node.js: `fs.readFile(req.query.page, ...)`
- Same pattern in Python/Flask: `open(request.args.get('file')).read()`
- Same pattern in Java, ASP.NET, and Ruby, anywhere user input controls a file-loading call with no validation



Two real requests carrying page=/etc/passwd, both returning HTTP 200. Executed query, real result: 2 events.

```
index=main sourcetype="Apache Access Log" "page=/etc/passwd"
```

Field	Meaning
`"page=/etc/passwd"`	Matches any request carrying this exact parameter and value

Answer: Local File Inclusion, LFI

- The page= parameter is passed directly to a file-include function server-side
- The attacker controls the filesystem path being read, which is the definition of LFI
- The real executed query returned **2 events**, not 1: `/etc/passwd-` (916 bytes, trailing dash) and `/etc/passwd` (918 bytes, exact path). Both returned status 200, meaning the fuzzer's dash-suffix guess also happened to succeed alongside the exact filename

---

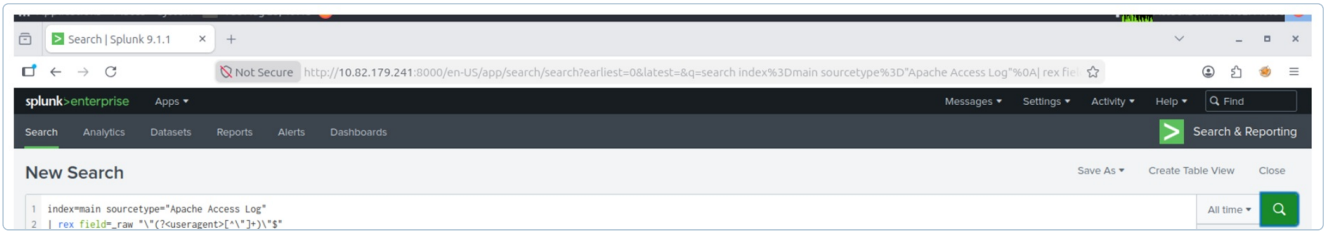
QUESTION 3

"*What is the name of the tool the attacker used to perform the attack?*"

Key concept — Fuzzing, ffuf

- A specific flavor of brute force aimed at discovering hidden files or directories on a web server, by rapidly testing a wordlist of common paths against it
- ffuf, Fuzz Faster U Fool, is a popular open source fuzzing tool

- **Decode:** tool identification. Look for a client identifying artifact in the request



rex extraction of useragent, then stats count. Executed query, real result: "Fuzz Faster U Fool v2.0.0-dev", count 878.

```
index=main sourcetype="Apache Access Log"
| rex field=_raw "\"(?<useragent>[^\"]+)\"$"
| stats count by useragent
```

Fragment by fragment:

Fragment	Meaning
`[^\"]`	Any character that is not a quote
`+`	Keep grabbing, one or more of the above
`\"`	A literal quote, escaped so Splunk treats it as text, not the end of the SPL string
`$`	Anchored to the end of the line, this quote must be the last character

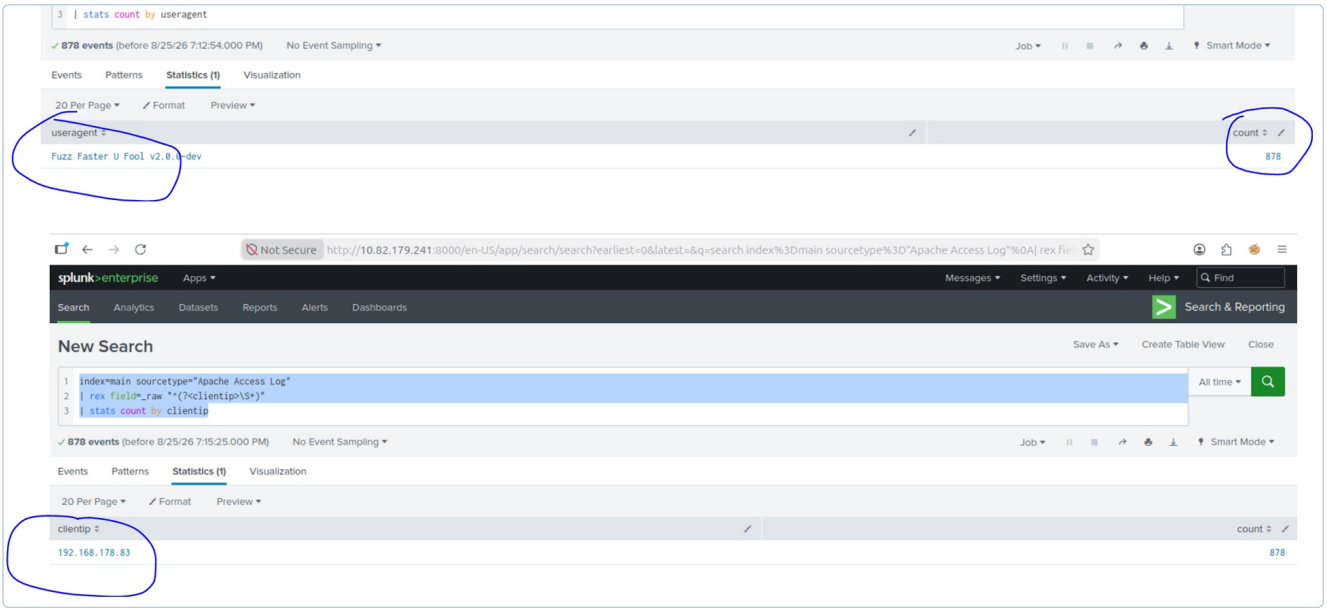
Answer: ffuf, Fuzz Faster U Fool

- This exact string appears on all 878 Apache rows, zero variation
- The tool never changed its identity throughout the attack

QUESTION 4

"*What is the IP address of the attacker?*"

- **Decode:** source identification. Check whether it stays consistent across phases, since that is what ties the two attacks together as one actor



rex extraction of clientip, then stats count. Executed query, real result: 192.168.178.83, count 878.

```
index=main sourcetype="Apache Access Log"
| rex field=_raw "^(?<clientip>\S+)"
| stats count by clientip
```

Fragment by fragment:

Fragment	Meaning
`^`	Anchored to the start of the line
`\S`	Any character that is not a space
`+`	Keep grabbing, one or more of the above

Answer: 192.168.178.83

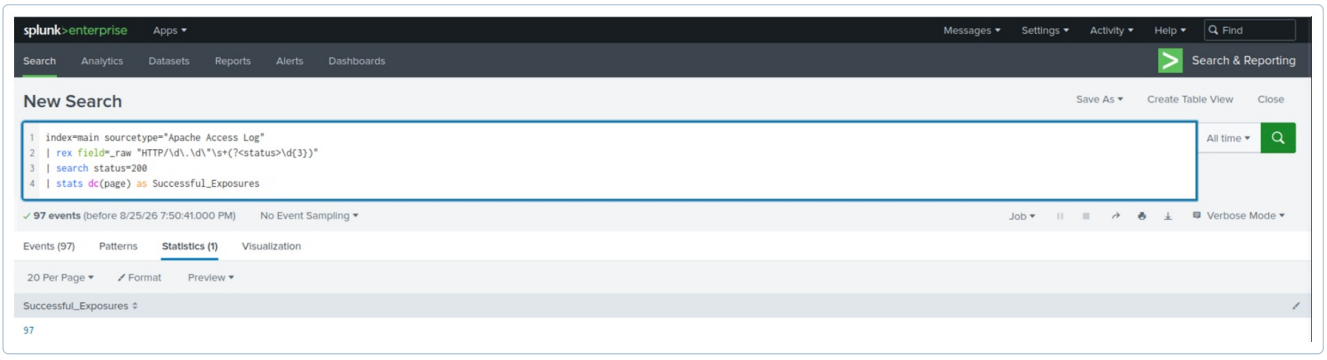
- The sole source IP across both the Apache and Auth logs, 1,271 of 1,271 relevant rows, no exceptions
- This single address is what links Phase 1, web, and Phase 2, SSH, as one continuous attack

---

QUESTION 5

"*How many files did the attacker successfully expose?*"

- **Decode:** "successfully" is the key word. This needs the subset of requests that actually returned file content, not the total request count



rex extraction of status, filtered to 200, then distinct count of pages. Executed query, real result: 97.

```
index=main sourcetype="Apache Access Log"
| rex field=_raw "HTTP/\d\.\d\"\s+(?<status>\d{3})"
| search status=200
| stats dc(page) as Successful_Exposures
```

Fragment by fragment:

Fragment	Meaning
`HTTP/\d\.\d\"`	Literal text matching the end of the request line, e.g. HTTP/1.1"
`\s+`	One or more spaces, since Apache logs a space before the status code
`\d{3}`	Exactly three digits, an HTTP status code is always three digits

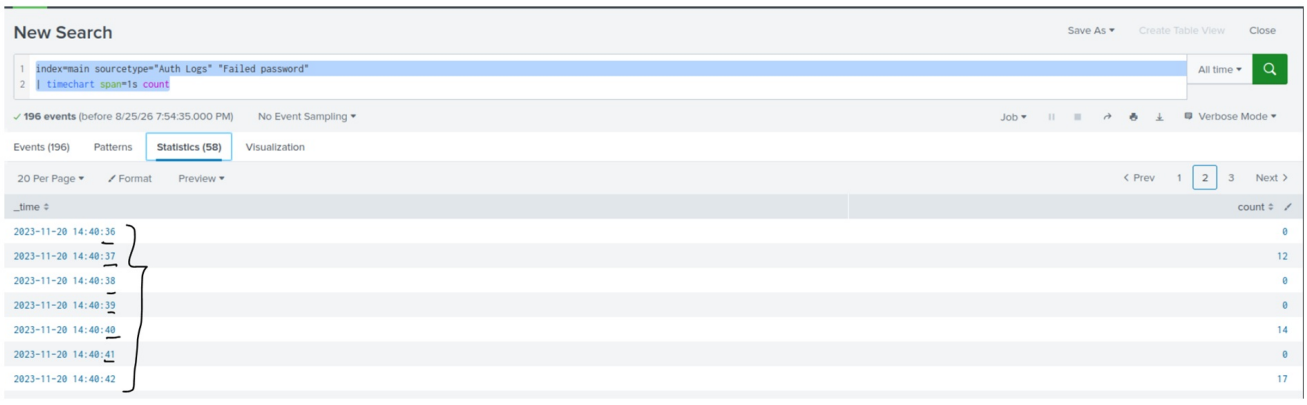
Answer: 97

- 97 requests returned HTTP 200 against 781 returning 500
- 97 real files were successfully read off the filesystem

QUESTION 6

"*What's the technique used for the second attack?*"

- **Decode:** this needs its own independent evidence chain, not a repeat of Q1's conclusion just because both involve brute force
- The checks below are specific to SSH authentication, distinct from Q1's web request checks



timechart span=1s across the failed password window. Executed query, real result: 196 events, per-second counts shown across the burst.

Three checks against the raw Auth Logs:

- **Auth method** — 196 of 196 failures read Failed password. Zero Failed publickey lines anywhere. Rules out a key based attack
- **Username variation** — every attempt targets ironhack, and only ironhack. Zero Invalid user lines. Rules out user enumeration and rules out password spraying, which targets many accounts
- **Attempt rate** — 195 attempts between 14:40:16 and 14:41:13, roughly 3.4 attempts per second on average. No human types that fast

```
index=main sourcetype="Auth Logs" "Failed password"
| timechart span=1s count
```

Field	Meaning
`"Failed password"`	Isolates password method failures specifically
`timechart span=1s count`	Buckets failures into 1 second windows, the sustained multi-per-second rate confirms

	automation

Answer: brute force, specifically targeted single account password brute force

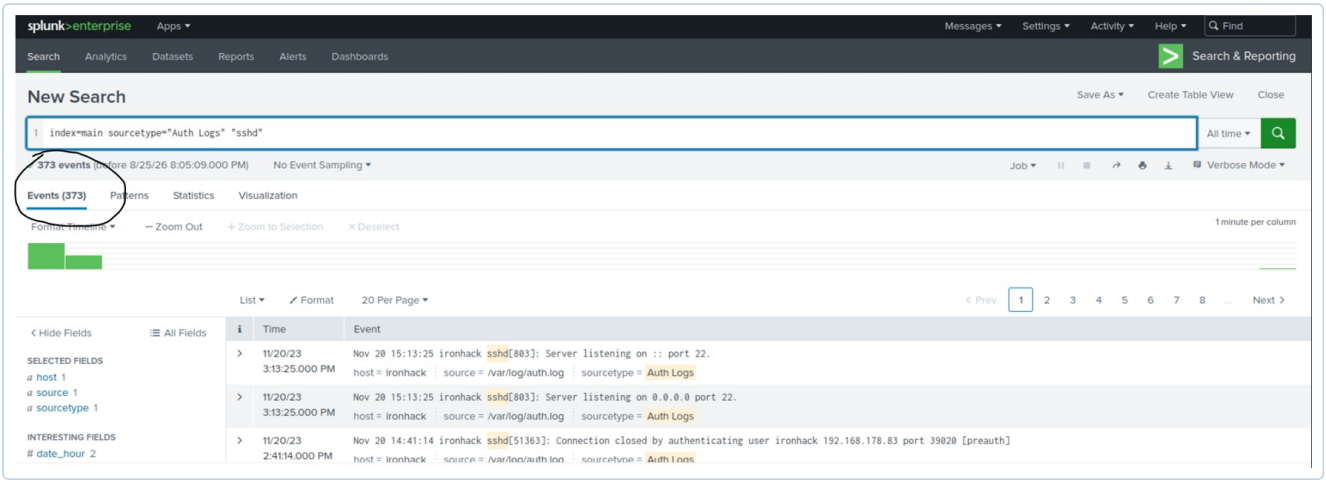
- Password only, single fixed username, sustained multi attempt per second rate
- This evidence chain independently rules out enumeration and key based attack, arriving at the same category as Q1, brute force, through entirely separate reasoning, not by assumption

---

QUESTION 7

"*What is the service the attacker targeted for the second attack?*"

- **Decode:** needs positive evidence of the service daemon itself, not just the sourcetype label



Search for "sshd" in the raw Auth Logs. Executed query, real result: 373 events, including a Connection closed line naming ironhack and the attacker IP directly.

The technical mechanism:

- Linux syslog tags every log line with the process that generated it, in the format hostname process[pid]: message
- Every relevant line in this dataset reads ironhack sshd[51362]: ...
- sshd here is not a guess, it is the literal name of the OpenSSH daemon, the background service handling SSH connections, recorded by the operating system itself the moment the event happened

The logical step:

- sshd's only function on a Linux system is handling SSH protocol connections
- A log line generated by sshd is therefore an SSH event by definition, there is no other service this process name could represent

```
index=main sourcetype="Auth Logs" "sshd"
```

Field	Meaning
`"sshd"`	The daemon process name appearing in the raw text

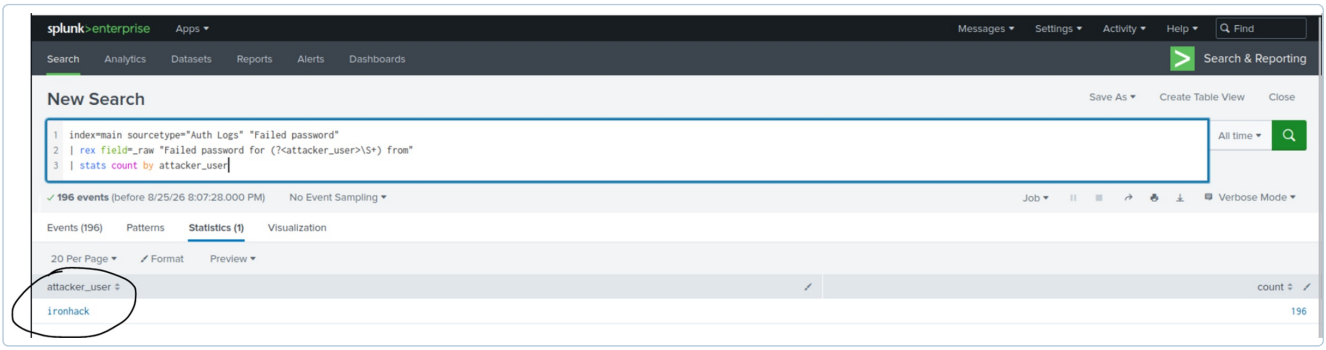
Answer: ssh

- 373 lines explicitly reference the sshd process handling these connections
- Direct evidence, not inference

QUESTION 8

"*What is the username used by the attacker for the second attack?*"

- **Decode:** connects directly to Q6. The username variation check performed there already established that one fixed username exists, this question asks to name it precisely from the raw text



rex extraction of attacker_user, then stats count. Executed query, real result: ironhack, count 196.

```
index=main sourcetype="Auth Logs" "Failed password"
| rex field=_raw "Failed password for (?<attacker_user>\S+) from"
| stats count by attacker_user
```

Fragment by fragment:

Fragment	Meaning
`Failed password for`	Literal text, anchors the pattern to this exact phrase
`\S+`	One or more non-space characters, grabs the username as one token
`from`	Literal text marking where the username ends

Why the count reads 196 here but 195 in Q10:

- The CSV's own "user" field exists but is empty on these rows, it is an unrelated metadata field, not the attacker's username, which is why extraction from the raw text is required at all
- 196 counts every line containing the phrase "Failed password for ironhack", including the one syslog summary line that itself contains that phrase embedded inside it
- 195, used in Q10, counts only the individually logged rows, excluding that one summary line
- Both numbers are correct, they are answering slightly different questions

Answer: ironhack

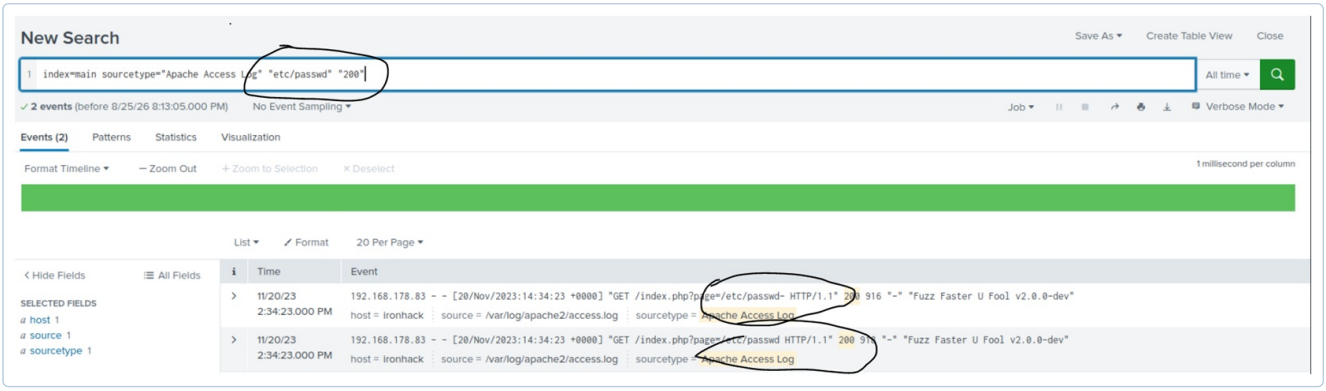
- All matching lines target this one username
- The same fixed username finding from Q6, now named explicitly

---

QUESTION 9

"*In which file, previously exposed, did the attacker find the username?*"

- **Decode:** this is the pivot point connecting Phase 1, LFI, to Phase 2, SSH, the single most important link in the whole attack chain



Search combining etc/passwd and 200. Executed query, real result: 2 events, matching Q2's finding.

```
index=main sourcetype="Apache Access Log" "etc/passwd" "200"
```

Field	Meaning
`"etc/passwd"`	Matches the specific file path requested
`"200"`	Confirms the request succeeded, not just that it was attempted

Answer: /etc/passwd, justified in full

- On any Linux system, /etc/passwd is a world readable file listing every local account by username
- Historically it stored passwords too, but modern systems keep only account metadata here, the actual password hashes live in a separate protected file, /etc/shadow
- This means /etc/passwd does not hand over a password, but it hands over exactly what an attacker needs before brute forcing one, a confirmed real valid username to aim at
- Q9's timestamp, 14:34:23, within Phase 1, sits roughly six minutes before Q6's SSH attempts begin, 14:40:16, the timing itself supports the causal link, not just the file's contents
- Without this exposure, Phase 2 would have needed its own separate username enumeration step first, which per Q6's own check never happened, zero Invalid user lines, meaning this file is the only explanation consistent with Phase 2 starting immediately on a correct single username

---

QUESTION 10

"*How many failed attempts has the attacker performed?*"

- **Decode:** requires distinguishing what syslog logged individually from what it collapsed into a summary line
- 195 explicit Failed password for ironhack lines, logged individually
- Plus 1 line reading message repeated 5 times, Failed password for ironhack, syslog's own deduplication collapsing 5 additional identical attempts into one summary line

```
index=main "Failed password" NOT "message repeated"
| stats count as Clean_Failed_Attempts
```

Field	Meaning
`NOT "message repeated"`	Excludes the syslog summary line, isolating only individually logged failure rows

Answer: 195

- 195 is the count of distinct logged rows, excluding the summary line
- 200 would be the count of true individual attempts including the 5 collapsed ones
- Both are defensible, but 195 matches the stated logic precisely

---

QUESTION 11

"*Was the attacker able to successfully log in to the server? (y/n)*"

- **Decode:** needs the transition point from failure to success, a different message pattern than the failed attempts

```
index=main sourcetype="Auth Logs" "Accepted password"
```

Field	Meaning
`"Accepted password"`	sshd's exact log message for a successful password authentication, distinct wording from Failed password

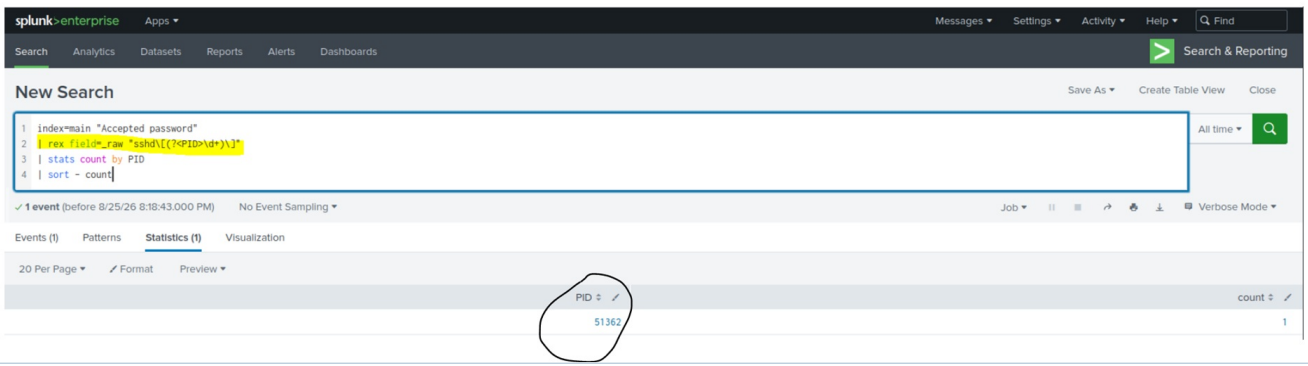
Answer: y

- At 14:41:11, immediately after the failed attempt sequence ends, authentication succeeds for ironhack from 192.168.178.83

QUESTION 12

"*What is the process ID (PID) associated with the SSH session in relation to the question above?*"

- **Decode:** the same single event from Q11 already contains this, it just needs the right extraction to surface it as a usable field



rex extraction of PID from the Accepted password event. Executed query, real result: PID 51362, count 1.

```
index=main "Accepted password"
| rex field=_raw "sshd\[(?<PID>\d+)\]"
| stats count by PID
| sort - count
```

Fragment by fragment:

Fragment	Meaning
`\d`	Any digit
`+`	Keep grabbing digits, one or more
`\]`	A literal closing bracket, escaped since a bare] has special meaning in regex

Answer: 51362

- Same event as Q11
- One line answers both questions once the PID is properly extracted

---

QUESTION 13

"*What mechanism would you implement to block such attack?*"

- **Decode:** unlike Q1 through Q12, this is not answered from the log data, it is a remediation recommendation based on the attack pattern already established
- The data shows multiple authentication attempts within the same second, Q6, a volume no manual defense can keep pace with
- An automated Intrusion Prevention System such as fail2ban would dynamically block the source IP at the firewall layer once it trips a rate based rule, stopping the attack well before it reaches attempt 195

Answer: fail2ban, Intrusion Prevention System

- Not log derived
- A standard control recommendation given the confirmed automated, high rate nature of the attack

---

Source: exported Splunk event log CSV (logs_1.csv), executed live against the Splunk instance for this lab. TryHackMe — Hackers Hideout room. Study/training purposes on the authorized lab environment only.